

Memory API in UNIX Systems

Allocating and Managing Memory in C

September 22, 2026

Types of Memory in C

- **Stack (Automatic) Memory:** Allocations and deallocations are managed implicitly by the compiler.
- Space is made upon function call and deallocated on return.
- **Heap Memory:** All allocations and deallocations are explicitly handled by the programmer, providing a way to store long-lived memory.

Code Example: Stack vs. Heap

```
// Automatically managed on the stack
int x = 5;

// Safely allocating on the heap
int *arr = malloc(10 * sizeof(int));
```

Allocating Memory: `malloc()`

- The `malloc(size_t size)` call takes the number of bytes needed and returns a pointer to the newly-allocated space, or `NULL` on failure.
- **Best Practice:** Use `sizeof()` as a compile-time operator instead of hardcoding sizes (e.g., `malloc(40)`).
- **String Allocation:** Use `strlen(s) + 1` to make room for the end-of-string character, as `sizeof()` will not work correctly for strings.

Code Example: Allocation Types

```
// Specific allocation for 10 integers
int *arr = malloc(10 * sizeof(int));

// +1 makes room for the null terminator
char *str = malloc(strlen(name) + 1);
```

Deallocating Memory: `free()`

- To free heap memory no longer in use, call `free()` and pass it the pointer returned by `malloc()`.
- The size of the allocated region is tracked internally by the memory-allocation library, so the user does not need to pass it.
- While the OS reclaims all process memory when a program exits, programmers should maintain the habit of explicitly freeing memory to avoid poor form.

Code Example: Proper Deallocation

```
int *arr = malloc(10 * sizeof(int));  
  
// Giving the memory back to the system  
free(arr);
```

Common Memory Errors

- **Segmentation Faults:** Passing unallocated pointers to routines like `strcpy()` causes illegal memory access and crashes.
- **Buffer Overflows:** Not allocating enough memory (e.g., forgetting the +1 for a string), which can silently overwrite contiguous blocks of important memory.
- **Memory Leaks:** Forgetting to free memory, which eats up RAM and eventually crashes the system.
- **Dangling Pointers:** Freeing memory too early or multiple times confuses the allocation library.

Code Example: Memory Leak

```
while(1) {  
    malloc(1024); // Classic example of a memory leak  
}
```

Underlying OS Support & Advanced APIs

- `malloc()` and `free()` are not system calls, but library calls built on top of OS system calls.
- **Heap Boundary Control:** System calls `brk()` and `sbrk()` extend or shrink the heap of the process in main memory.
- You should never call these directly; the libraries manage it well.
- **Virtual RAM / Swap:** The `mmap(...)` call can map anonymous memory to swap space to prevent your system from crashing if it runs out of main memory.
- **Advanced Library Functions:**
 - `calloc()`: Allocates memory and zeroes it out.
 - `realloc()`: Resizes an existing memory region.